A_Spectra

April 21, 2026

1 Project 1: Time-frequency analysis - Part A: Spectral Estimation

:::{important} Learning Goals - [ ] I understand the basic concept of spectral estimation. - [ ] I can de-noise data using filtering. - [ ] I understand the concept of Aliasing :::

1.0.1 Techniques:

- Fast Fourier Transforms

1.1 Table of Contents

1.1.1

1.1.2

1.1.3

1.1.4

(h-mono)= # Monofrequent signals

A monofrequent signal can be described by a sine wave of a given frequency f, amplitude A and phase ϕ

$y(t) = A\sin(f \cdot t + \phi)$

Electrical engineers and seismologists will prefer the complex version

$y(t) = A\exp\left(\mathrm{i}(2\pi f \cdot t + \phi)\right).$

- The frequency has the inverse unit of the time series $y(t)$.
- The frequency f can also be be expressed as the angular frequency $\omega = 2\pi f$, which describes the number of radians per second. $y(t) = Ae^{\mathrm{i}(-\omega t+\phi)}$

(h-ex1)= ## Widget 1: Monochromatic signals :::{tip} *Exercise 1a:* Monochromatic signals in time domain **Run the cell below** and explore the effects of varying - frequency f - amplitude A (parameter amp) - phase of the signal

on the **time series (left)**. :::

Hint: Do not increase the frequency too much early on.

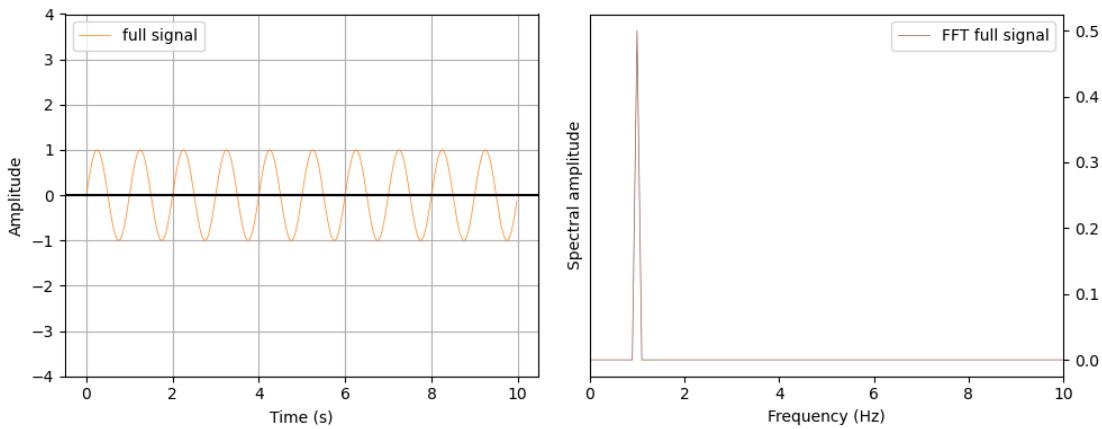
:::{tip} *Exercise 1b:* Monochromatic signals in spectral domain. **Run the cell below again and focus on the spectrum plotted on the right.** - Why does changing the phase of the signal have no effect on the spectrum? - Add white noise to the signal (parameter *noiselevel*). Why does the spectral peak stand out even in a signal that is not recognizable in time domain?

[1]:
```
import ipywidgets as widgets
import numpy as np
from functions.aliasing import single_sine_wave, open_figures
%matplotlib widget

ax_time, ax_freq = open_figures()
info_leak = widgets.HTML()

_ = widgets.interact(single_sine_wave,
                     ax_freq=widgets.fixed(ax_freq),
                     ax_time=widgets.fixed(ax_time),
                     f=(0.1, 25, .1),
                     amp=(0, 4, .1),
                     phase=(0, 360, 0.1),
                     noiselevel=widgets.fixed(0.0),
                     drift=widgets.fixed(0.0), #(0, 2, 0.1),
                     pulse=widgets.fixed(0.0), #(0, 2, 0.1),
                     plot_2nd_trace=widgets.fixed(False),
                     apply_lowpass=widgets.fixed(False),
                     quantization_bits=widgets.fixed(32),
                     freq_lowpass=widgets.fixed(1.)
                     )
```

```
interactive(children=(FloatSlider(value=1.0, description='f', max=25.0, min=0.
↪1), FloatSlider(value=1.0, descr…
```



1.2 _____

(h-ex2)= ## Widget 2: Noise and Quantization :::{tip} *Exercise 2:* Monochromatic signals in spectral domain. **Run the cell below again and focus on the spectrum plotted on the right.** - Add white noise to the signal (parameter *noiselevel*). Why does the spectral peak stand out even in a signal that is not recognizable in time domain? - Describe the effect of reducing the number of bits for quantization. Compare the noise level induced by quantization to the values in the table on slide 11. - For a signal with amplitude 2 and a quantization with 8 Bit, what is the maximum noise level at which the quantization dominates the overall noise?

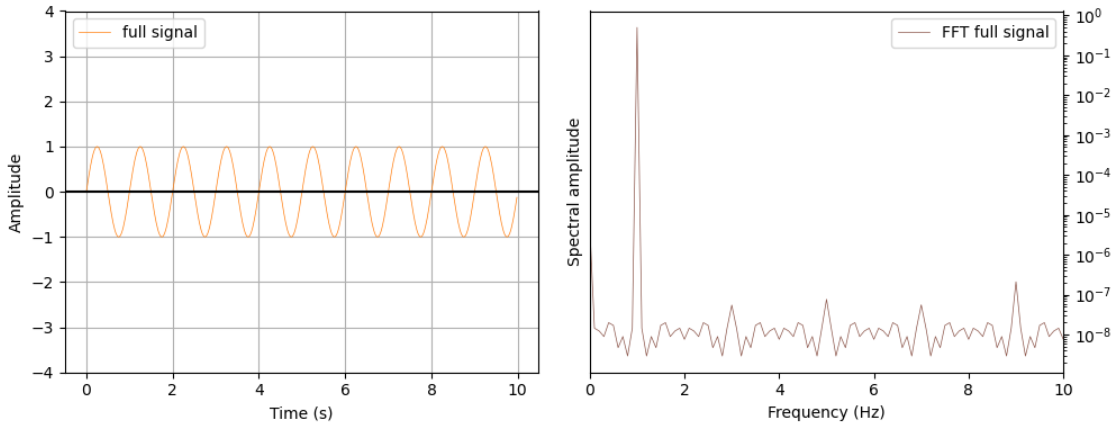
[2]:
```
import ipywidgets as widgets
import numpy as np
from functions.aliasing import single_sine_wave, open_figures
%matplotlib widget

ax_time, ax_freq = open_figures()
ax_freq.set_yscale('log')
info_leak = widgets.HTML()

_ = widgets.interact(single_sine_wave,
                     ax_freq=widgets.fixed(ax_freq),
                     ax_time=widgets.fixed(ax_time),
                     f=(0.1, 25, .1),
                     amp=(0, 4, .1),
                     phase=(0, 360, 0.1),
                     noiselevel=(0, 1, 0.001),
                     drift=widgets.fixed(0.0), #(0, 2, 0.1),
                     pulse=widgets.fixed(0.0), #(0, 2, 0.1),
                     plot_2nd_trace=widgets.fixed(False),
                     apply_lowpass=widgets.fixed(False),
                     quantization_bits=(4, 32, 1),
                     freq_lowpass=widgets.fixed(1.)
```

```
                        )
```

```
interactive(children=(FloatSlider(value=1.0, description='f', max=25.0, min=0.
↪1), FloatSlider(value=1.0, descr…
```



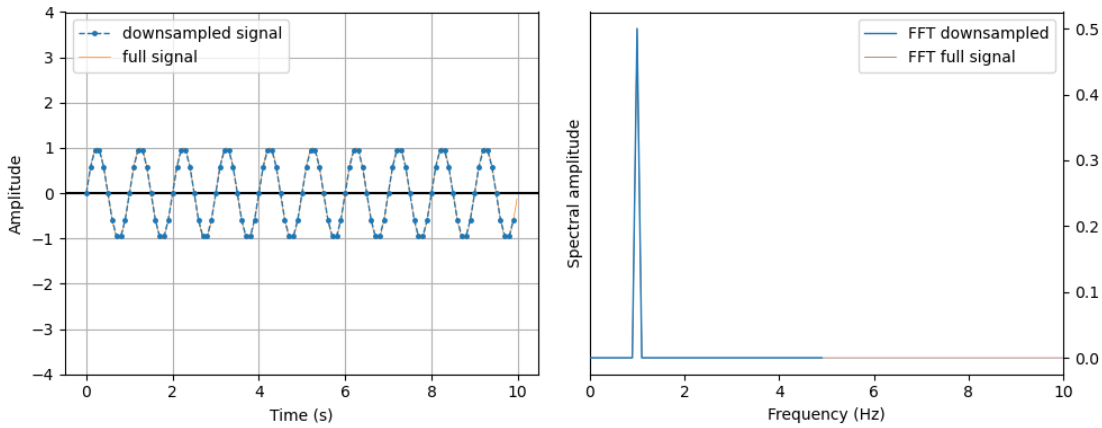
(h-ex2)= ## Widget 3: Downsampling, Aliasing and the Shannon theorem :::{tip} *Exercise 3:* Downsampling Regard the next code cell: It contains a second trace (blue), which is plotting only every **fifth** sample of the original trace (orange). - Increase the frequency carefully towards 5 Hz. What do you observe in the blue time series? - What do you observe in the spectrum? - What happens when you increase the frequency above 10 Hz? - What is the effect of the lowpass pre-filter? - Why is the effect of the LP not perfect, if set to the Nyquist frequency of 5 Hz? :::

[3]:
```
import numpy as np
from functions.aliasing import single_sine_wave, open_figures
%matplotlib widget

ax_time, ax_freq = open_figures(f_nyquist=5.)

_ = widgets.interact(single_sine_wave,
                     ax_freq=widgets.fixed(ax_freq),
                     ax_time=widgets.fixed(ax_time),
                     f=(0.2, 20, .2),
                     amp=(0, 4, .1),
                     phase=(0, 360., 0.1),
                     noiselevel=widgets.fixed(0.0),
                     drift=widgets.fixed(0.0),
                     pulse=widgets.fixed(0.0),
                     plot_2nd_trace=widgets.fixed(True),
                     apply_lowpass=False,
                     quantization_bits=widgets.fixed(32),
                     freq_lowpass=(0.05, 20., 0.05),
                     )
```

```
interactive(children=(FloatSlider(value=1.0, description='f', max=20.0, min=0.2,␣
↪step=0.2), FloatSlider(value=…
```



(h-quiz)= ## Quiz: Effects of high-frequency noise :::{tip} *Quiz:* Relation between original and aliased frequency - Increase the frequency to 2.5 Hz and estimate the frequency of the blue trace from the time series. Which value? - Increase the frequency to 7.5 Hz and estimate the frequency of the blue trace from the time series. Which value? - Increase the frequency to 19 Hz and estimate the frequency of the blue trace from the time series. Which value? - **Next, without running it**: If you would increase the frequency to 16 Hz, what will be the frequency of the aliased signal?

[2]:
```
# Easiast way to add an iframe and control the width (first number) and height␣
↪(second number) parameters is by using Python directly
from IPython.display import IFrame
IFrame('https://speakup.epfl.ch/room/join/57188', 400, 1000)
```

[2]:
```
<IPython.lib.display.IFrame at 0x15cda5f40>
```

(h-aliasing)= # Aliasing

Spectral components in a signal can be recovered by continuous sampling up to a frequency of *half* the sampling rate. This frequency is called the *Nyquist frequency* f_N. The **Nyquist–Shannon sampling theorem** requires that the frequency range of a signal does not exceed the Nyquist frequency, if the signal is to be reconstructed using digital signal processing.

Frequency components higher than f_N will be mapped into the frequency range $(0, f_N)$. :::::{hint} Aliasing :class: dropdown **Aliasing** is a process that can (generally) not be reversed after digitization of the signal.

The Moiré effect is a spatial aliasing effect caused by sampling an image with regular high-frequency structure with a similar, but lower resolution.

:::{figure} imgs/Moiré_pattern.png :name: teaching ideas An illustration of a Moiré pattern when taking a picture of an LED screen.

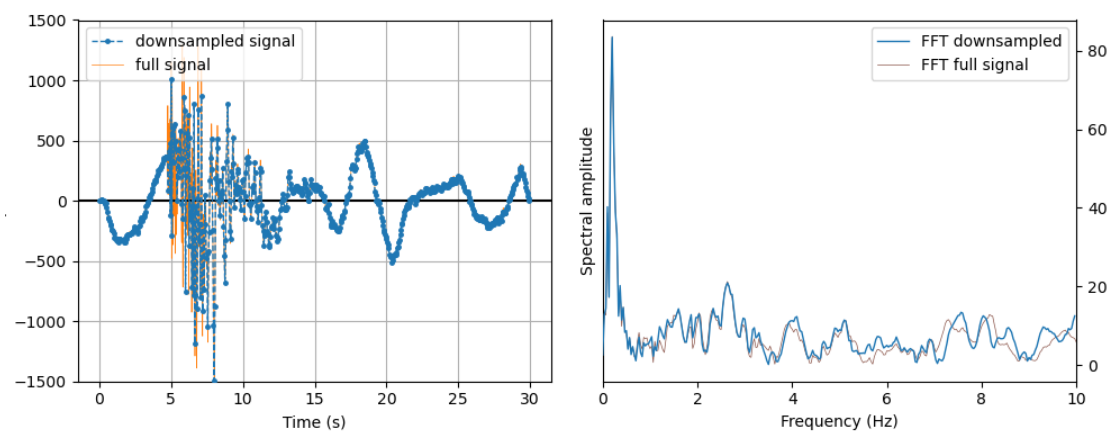
(h-ex3)= ## Widget 4: Effects of high-frequency noise :::{tip} *Exercise 4:* Noise Regard the next

code cell. It shows a short seismic signal in the presence of various noise sources. By increasing the value of noiselevel, you can add noise that is outside the frequency range of the downsampled signal.. - Why does a peak at 3.33 Hz appear in the spectrum of the downsampled data (FFT), that is not present in the full bandwidth data (FFT full)? - What is the original frequency of this peak? What could be the source of this noise? - Try to suppress it using a low-pass filter. :::

[14]:
```
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from functions.aliasing import seismic_signal, open_figures
%matplotlib widget

ax_time, ax_freq = open_figures(f_nyquist=5.)
ax_time.set_ylim(-1500, 1500)
# TODO: Arrange controls nicer. Maybe use interactive_output with a pre-defined␣
↪HBox
_ = widgets.interact(seismic_signal,
                     ax_freq=widgets.fixed(ax_freq),
                     ax_time=widgets.fixed(ax_time),
                     add_noise=False,
                     noiselevel=(0., 2., 0.1),
                     drift=(0, 2, 0.1),
                     pulse=(0, 2, 0.1),
                     apply_lowpass=False,
                     freq_lowpass=(0., 40., 0.2),
                     welch=widgets.fixed(False),
                     winlen_sec=widgets.fixed(1.),
                     )
```

```
interactive(children=(Checkbox(value=False, description='apply_lowpass'),␣
↪FloatSlider(value=5.0, description='…
```



:::{important} Aliasing A solution for the problem of Aliasing is to low-pass filter the signal before

digitization.

This filter needs to be implemented in analog electronics before the sampling step!

High-frequency noise is almost always present in real data and will reduce the signal-to-noise ratio significantly, if not handled before digitization. :::

(h-ex4)= ## Widget 5: Aliasing guessing game :::{tip} *Exercise 4:* The following signal contains several spectral components. At which frequencies?

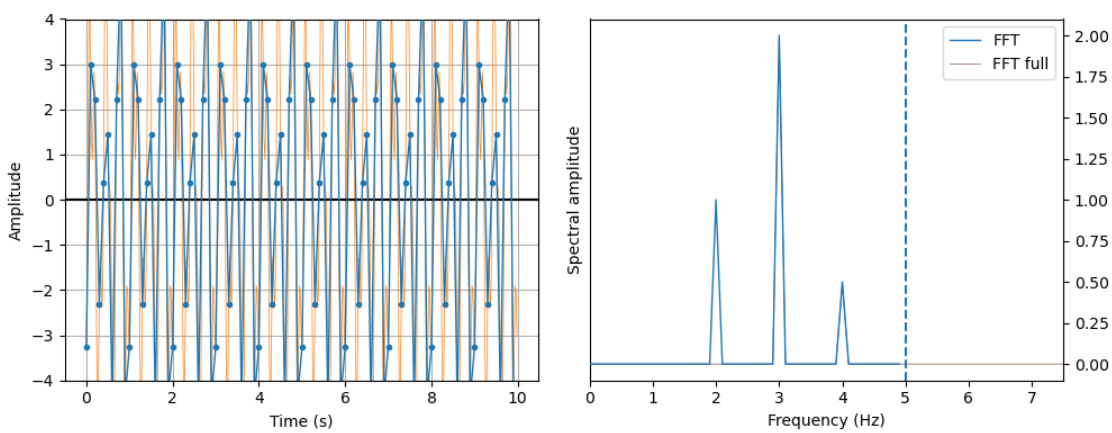
- Try to estimate first without clicking any buttons.
- Apply the lowpass filter with different cut-off frequencies and estimate again.
- Why does the signal change at all when applying a low-pass filter at 5 Hz? :::

[15]:
```
from functions.aliasing import plot_n_monochromatic_signals, open_figures
import ipywidgets as widgets
import matplotlib.pyplot as plt
import numpy as np

ax_time, ax_freq = open_figures(f_nyquist=5.)

_ = widgets.interact(plot_n_monochromatic_signals,
                 ax_time=widgets.fixed(ax_time),
                 ax_freq=widgets.fixed(ax_freq),
                  apply_lowpass=False,
                  freq_lowpass=(0., 10., 0.1)
                  )
```

```
interactive(children=(Checkbox(value=False, description='apply_lowpass'),␣
↪FloatSlider(value=5.0, description='…
```



(h-real)= # Real data _____

(h-ex5)= ## Exercise 5: Time-dependent frequency content :::{tip} *Exercise 5:* Spectral content **Most real signals have a varying spectral content**. The next widget presents the signal of a

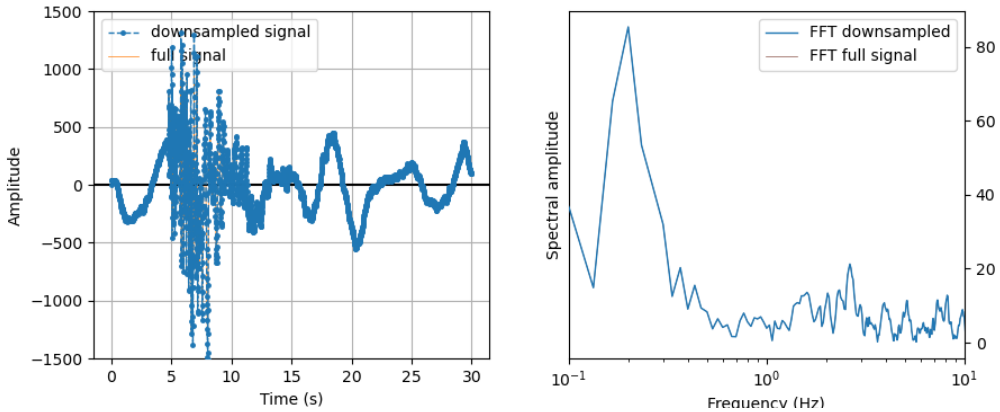
small Earthquake in about 50 km distance recorded in Bavaria. Use the widget below to test the spectral content of the seismic signal presented here. - What is the frequency range of the seismic signal? - What is the frequency range of the background noise? - Is the background noise present during the earthquake? - Can the background noise be suppressed without affecting the arrival time of the earthquake signal? :::

[16]:
```
from functions.aliasing import seismic_signal_time_window
import ipywidgets as widgets
import matplotlib.pyplot as plt
%matplotlib widget
plt.close('all')
# set up plot to speed up execution time later
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))
ax_time = ax[0]
ax_freq = ax[1]
ax_time.set_ylim([-1500, 1500])
ax_time.grid(True)

ax_time.set_xlabel('Time (s)')
ax_freq.set_xlabel('Frequency (Hz)')
ax_time.set_ylabel('Amplitude')
ax_freq.set_ylabel('Spectral amplitude')
ax_freq.set_xscale('log')
ax_freq.set_xlim(0.1, 10)
ax_freq.yaxis.tick_right()

_ = widgets.interact(seismic_signal_time_window,
                     ax_freq=widgets.fixed(ax_freq),
                     ax_time=widgets.fixed(ax_time),
                     freq_highpass=(0.01, 40, 0.05),
                     freq_lowpass=(0.25, 40, 0.25),
                     t_start=(0., 30.),
                     t_end=(0., 30.),
                     welch=widgets.fixed(False),
                     winlen_sec=widgets.fixed(10.)
                 )
```

```
interactive(children=(FloatSlider(value=0.0, description='t_start', max=30.0),␣
↪FloatSlider(value=30.0, descrip…
```



[17]:
```
from IPython.display import IFrame
IFrame('https://speakup.epfl.ch/room/join/53958', 400, 300)
```

[17]: <IPython.lib.display.IFrame at 0x7f2ada4b9a60>